

ShipShape — Audit Report

Project: Ship (US-Department-of-the-Treasury/ship) **Branch:** shipshape/audit **Author:** Tyler Xia **Audit window:** Mon 2026-05-18 → Tue 2026-05-19 23:59 (hard gate) **Scope:** 7 categories — baseline measurements only. No fixes here; treatment goes in implementation branches.

This report's mission is diagnosis. Each category section follows the brief's template: methodology, numbers, top findings ranked by impact, and the improvement target derived from the brief.

Reading order for reviewers: Skip to [§ Summary](#) for the at-a-glance table; jump to a category section for raw numbers and methodology. Raw output files (axe JSON, Lighthouse reports, EXPLAIN ANALYZE dumps, autocannon JSON) live in [shipshape/audit/raw/](#).

Summary

#	Category	Baseline (headline)	Improvement target
1	Type Safety	any: 260 , as <T>: 460 , !: 324 , @ts-*--error: 1 . Strict mode on in api/shared; web has stock strict: true only — turning on root's superset surfaces ~ 102 hidden errors across 22 files .	Eliminate 25% of violations w/ correct narrowing
2	Bundle Size	Total web/dist: 3.8 MB . Total JS: ~ 2.15 MB . Largest chunk: 2,073,698 bytes (1.98 MB, 92% of all JS) in index-*.js — single bundle, Vite warns about it.	-15% total or -20% initial via code split
3	API Response Time	/api/issues P99 grows 165→515ms across c=10→50 (102 KB response per call). /api/auth/me melts at c=50 (P99 1575ms , max 2785ms) due to pool saturation. /api/dashboard/my-work P95 6× at c=25 vs c=10.	-20% P95 on ≥2 endpoints
4	DB Query Efficiency	At seeded volume (104 issues, 250 docs) most plans run in 1–2 ms. Correlated subquery in projects-with-inferred-status hits 596 buffer hits . JSONB-key filters (properties->>'assignee_id')::uuid + properties->>'state' use seq-scan-and-filter — no functional indexes.	-20% queries on a flow or -50% on slowest
5	Test Coverage & Quality	866 E2E (71 files), 447 API unit (28 files), 151 web unit (16 files) — total ~1,464	+3 meaningful tests on untested paths, or fix 3 flakes w/ RCA
6	Runtime Errors	Stack trace leaked to client on malformed JSON (Express default handler). 3 of 7 malformed-input probes succeeded silently. XSS-shaped title accepted verbatim. API tests slow-fail 54 min when Postgres is down.	3 fixes, ≥1 real data-loss scenario
7	Accessibility	5 of 12 routes have a serious violation: color-contrast across 46 total nodes ON dashboard, my-week, projects, team/allocation, team/status. 0 critical, 0 moderate, 0 minor. 7 routes are clean. Directly contradicts the WCAG 2.1 AA badge.	+10 Lighthouse on worst page or all Critical/Serious on top 3

Category 1 — Type Safety

Methodology

- **Static counts via Grep (ripgrep)** across `api/src/**/*.ts` , `web/src/**/*.ts` , `shared/src/**/*.ts` .
- Patterns:
 - `any` : `:\s*any\b|\bany\[<any>|as any\b`
 - `as <Type>` : `\bas\s+[A-Z]` (excludes `as const` , `as default` , `as unknown`)
 - `Non-null !` : `[a-zA-Z0-9_\]\]!(\.\|\\(|;|,|\)|\s*\}|\\s*$)` (covers `arr[0]!` , `foo()!` , `userId!` ; , `userId!.x`)
 - `@ts-* -error` : `@ts-ignore|@ts-expect-error|@ts-nocheck`
- **Strict-mode investigation:** Read all four `tsconfig.json` s — `root` + `api` + `web` + `shared`. Checked which inherit from which.
- **Web strict probe:** Created [web/tsconfig.strict-probe.json](#) extending `tsconfig.json` and adding `noUncheckedIndexedAccess` , `noImplicitReturns` , `noFallthroughCasesInSwitch` . Pending `tsc` run after `pnpm install` completes.

Baseline

Metric	Count	Top contributors
any (api)	229 across 23 files	tests: <code>accountability.test.ts</code> :32, <code>transformIssueLinks.test.ts</code> :37, <code>auth.test.ts</code> :24; code: <code>projects.ts</code> :13, <code>yjsConverter.ts</code> :12, <code>weeks.ts</code> :10
any (web)	31 across 8 files	<code>FileAttachment.tsx</code> :7, <code>SlashCommands.tsx</code> :6, <code>AIscoringDisplay.tsx</code> :6
any (shared)	0	clean
any total	260	
as <Type> (api)	250 across 26 files	<code>weeks.ts</code> :48, <code>issues.ts</code> :34, <code>projects.ts</code> :26, <code>team.ts</code> :25, <code>admin.ts</code> :14
as <Type> (web)	210 across 78 files	<code>UnifiedEditor.tsx</code> :25, <code>PropertiesPanel.tsx</code> :13, <code>DiffViewer.tsx</code> :8
as <Type> total	460	
Non-null ! (api)	290 across 27 files	<code>weeks.ts</code> :48, <code>issues.ts</code> :34, <code>team.ts</code> :28, <code>projects.ts</code> :26
Non-null ! (web)	34 across 19 files	<code>ReviewsPage.tsx</code> :5, <code>MyWeekPage.tsx</code> :4, <code>useSessionTimeout.ts</code> :3
Non-null ! total	324	
@ts-ignore / @ts-expect-	1	only <code>web/src/components/icons/uswds/Icon.test.tsx</code>

error / @ts-nocheck		
Strict mode enabled?	Yes (partial)	api+shared inherit root's strict + noUncheckedIndexedAccess + noImplicitReturns + noFallthroughCasesInSwitch; web has only stock strict: true (does not extend root config)
Strict-probe error count (web)	102 errors across 22 files	After adding noUncheckedIndexedAccess + noImplicitReturns + noFallthroughCasesInSwitch to web. Raw output: raw/web-strict-probe-errors.txt
Error breakdown by TS code	TS2532 × 41 (Object possibly undefined), TS18048 × 29 (possibly undefined), TS2322 × 12 (assignability), TS2345 × 11 (param type), TS7030 × 8 (missing return), TS18047 × 1 (possibly null)	~80 of 102 (78%) are real undefined-access bugs the lint doesn't catch today
Top files (strict-probe)	CommandPalette.tsx (13), lib/cn.ts (12), hooks/useSelection.ts (12), editor/CommentDisplay.tsx (12), editor/AIScoringDisplay.tsx (12)	Five files account for 61 of 102 errors

Top 5 violation-dense files

Combined (any + as <T> + !):

File	any	as	!	Total	Why it's problematic
api/src/routes/weeks.ts	10	48	48	106	Largest weekly-plan handler; mixes typed zod schemas with any rows from pool.query. Hot path.
api/src/routes/issues.ts	5	34	34	73	Issue CRUD + state transitions. any here makes state-machine bugs invisible to TS.
api/src/routes/projects.ts	13	26	26	65	Project ICE/RACI logic. properties JSONB cast to any instead of ProjectProperties.
api/src/routes/team.ts	1	25	28	54	Person/membership joins; ! on every req.userId is the dominant pattern.
api/src/routes/programs.ts	2	18	18	38	Same JSONB-cast pattern as projects.ts.

Note: many ! counts equal as counts because both reflect the same pattern of unsafe-cast-then-non-null on raw query rows. Fixing the underlying typing of pool.query would knock out both in one go.

Findings

1. **Web tsconfig.json does not extend the root** ([web/tsconfig.json](#)). Root sets strict + noUncheckedIndexedAccess + noImplicitReturns + noFallthroughCasesInSwitch . Web has only strict: true . **Impact:** the strongest TS constraints are silently disabled in the package with the most surface area (78 files using as , 19 using !). (Severity: high — fix surfaces real undefined-access bugs.)

- 2. **Untyped pool.query rows** propagate `any` through route handlers (visible in `documents.ts:18` — `Promise<{ canAccess: boolean; doc: any | null }>`). Each route then `as`-casts back to a `properties` type. **Impact:** ~250 `as <T>` in api are mostly this. A generic `query<T>` wrapper that infers the row type from a zod schema or a `Document<T>` discriminated row type would eliminate hundreds. (Severity: high — single change with broad reach.)
- 3. **Zod schemas drift from reality.** `createDocumentSchema` in `documents.ts` still accepts `program_id` and `sprint_id` even though migrations 027/029 dropped those columns. **Impact:** the API contract published in the OpenAPI spec lies about what fields are persisted. (Severity: medium — bugs at the schema/DB boundary.)
- 4. **Tests dominate any totals** (~155 of 260). This is acceptable in unit-test fixtures but inflates the headline number. The Category target should be measured on non-test files only: ~105 `any` in production code, of which ~80 in api routes. (Severity: methodological — affects the target denominator.)
- 5. **@ts-ignore is essentially absent (1 total).** Good sign — the team prefers refactoring to suppression. (Severity: positive finding.)

Improvement target (per brief)

Eliminate 25% of type safety violations with correct narrowing. Replacing `any` with `unknown` and leaving it unnarrowed does not count.

Target on the non-test set: drop 25% of (`any` + `as <T>` + `!`) in non-test files. With the proposed `pool.query<T>` generic, the `web/tsconfig` strict bump, and fixing the zod-schema drift, this is achievable in a few focused commits.

Category 2 — Bundle Size

Methodology

- Run `pnpm build` (which runs `pnpm build:shared && tsc && vite build` for the web package).
- Measure `web/dist/` size (total + per-chunk + per-asset).
- Add `rollup-plugin-visualizer` as a devDep on the audit branch only, re-build, capture treemap as `shipshape/audit/raw/bundle-stats.html`.
- Cross-reference `web/package.json` dependencies vs. actual imports for dead-deps.

Baseline

Metric	Value
Total web/dist size	3.8 MB
Total assets dir size	2.7 MB
Largest chunk	<code>index-C2vAyoQ1.js</code> — 2,073,698 bytes (1.98 MB raw, 589.49 KB gzipped)
Largest chunk share of JS	92% of total JS
Number of JS chunks (assets/*.js)	~145 (mostly small USWDS icons)
Total JS bytes	2,250,445 bytes (~2.15 MB)
Total CSS bytes	66,512 bytes (~65 KB, single file)
2nd-largest JS chunk	<code>ProgramWeeksTab-*.js</code> — 16,761 bytes (16.4 KB)

Chunks $\geq$ 100 KB	1 (the main one)
Vite build warning	"Some chunks are larger than 500 kB after minification. Consider dynamic import or <code>manualChunks</code> "

Treemap available at [raw/bundle-stats.html](https://raw.githubusercontent.com/rollup/rollup-plugin-visualizer/master/rollup-plugin-visualizer.html); composition summary at [raw/bundle-composition.txt](https://raw.githubusercontent.com/rollup/rollup-plugin-visualizer/master/rollup-plugin-visualizer.txt).

Bundle composition (top contributors)

Generated by `rollup-plugin-visualizer` with `ANALYZE=1 vite build`. Sizes here are pre-minification source sizes ("renderedLength" from the visualizer); the actual minified+chunked output is the 2.07 MB main bundle.

#	Package	Rendered	Gzip	Where it's used
1	emoji-picker-react	399.6 KB	72.4 KB	One component only (EmojiPicker.tsx) → ideal lazy-load
2	highlight.js	377.9 KB	118.5 KB	Only when rendering code blocks (via extension-code-block-lowlight) → ideal lazy-load
3	yjs	264.9 KB	55.5 KB	Editor pages only
4	prosemirror-view	236.3 KB	57.2 KB	Editor only
5	TipTap core	181.2 KB	36.9 KB	Editor only
6	react-dom	131.7 KB	42.4 KB	Universal (cannot split)
7	prosemirror-model	121.2 KB	28.7 KB	Editor only
8	lib0	106.5 KB	34.0 KB	Yjs dependency
10	diff-match-patch	80.6 KB	18.1 KB	One component only (DiffViewer.tsx) → ideal lazy-load
11	@tiptap/extension-code-block-lowlight	80.0 KB	23.3 KB	Pulls highlight.js
13	react-router	79.6 KB	19.5 KB	Universal
...	App code total	1,316.6 KB	(174 files)	<i>includes all editor pages, sidebars, components</i>

The editor stack (`@tiptap/*` , `prosemirror-*` , `yjs` , `lib0` , `y-prosemirror` , `lowlight` , `highlight.js`) sums to **~1.5 MB rendered** and is loaded on every route — including `/login` , `/team/directory` , `/team/status` , `/admin` , etc. that don't need an editor.

Findings (from `rollup-plugin-visualizer` `treemap`)

I added `rollup-plugin-visualizer` as a devDep (gated by `ANALYZE=1` env so it never runs in normal builds) and generated [raw/bundle-stats.html](#) plus [raw/bundle-top-contributors.txt](#).

Top contributors to the main `index-*.js` chunk (rendered raw bytes — full chunk total before minification is ~4.55 MB; the post-minification figure of 2.07 MB cited above is the actually-shipped size):

Bucket	Rendered	Gzip	% of chunk	Notes
app/src (web)	1,222 KB	270 KB	26.8%	application code
emoji-picker-react	409 KB	74 KB	9.0%	only used in <code>EmojiPicker.tsx</code>
highlight.js	387 KB	121 KB	8.5%	39 language files — only for code blocks
yjs	271 KB	57 KB	6.0%	core editor sync
prosemirror-view	242 KB	59 KB	5.3%	core editor
@tiptap/core	186 KB	38 KB	4.1%	core editor
react-dom	135 KB	43 KB	3.0%	framework
prosemirror-model	124 KB	29 KB	2.7%	core editor
lib0	109 KB	35 KB	2.4%	yjs dependency, 37 files
@dnd-kit/core	103 KB	22 KB	2.3%	drag-and-drop
diff-match-patch	82 KB	19 KB	1.8%	only used in <code>DiffViewer.tsx</code>
@tiptap/extension-code-block-lowlight	82 KB	24 KB	1.8%	wraps <code>highlight.js</code>
@popperjs/core	59 KB	23 KB	1.3%	tippy.js dep
react-router	81 KB	20 KB	1.8%	routing

Three obvious code-split wins worth a single targeted commit each:

1. **emoji-picker-react** (409 KB rendered) — lazy-load `EmojiPicker.tsx` .
2. **highlight.js** / **@tiptap/extension-code-block-lowlight** (469 KB rendered combined) — lazy-load the code-block extension; most documents have no code blocks.
3. **diff-match-patch** (82 KB rendered) — lazy-load `DiffViewer.tsx` (used only in revision-history views).

Together: **~960 KB rendered / ~218 KB gzipped removed from initial load** without any feature loss. Plus a `manualChunks` for `react-dom` + `react-router` + `@tanstack/query` as a long-lived vendor chunk for cache stability.

Improvement target

-15% total or implement code splitting that reduces initial load by -20%.

The three lazy-loads above + a vendor split should comfortably exceed -20% initial load reduction.

Category 3 — API Response Time

Methodology

- **Seed data:** Use `pnpm db:seed` plus a top-up script to hit the brief's targets — 500+ documents, 100+ issues, 20+ users, 10+ sprints. The standard seed creates a handful of demo entities; will write `shipshape/audit/scripts/seed-load.ts` if needed.
- **Endpoint selection:** Trace the React Query calls fired during five user flows (load /my-week, open a doc, list issues, load a sprint board, search). Top 5 by frequency become the benchmark set.
- **Tool:** `autocannon` via `corepack pnpm dlx autocannon` (no global install needed). Capture P50/P95/P99 at 10, 25, 50 concurrent connections per endpoint. JSON output → `shipshape/audit/raw/perf-<endpoint>-<conn>.json`.
- **Ambient conditions:** identical machine state for before/after — note CPU/memory/free-RAM at run time.

Baseline

Seed: 11 users, 5 programs, 15 projects, 35 weeks, **104 issues**, ~50 misc docs = ~250 documents total. Lighter than the brief's 500+ target but representative of real usage. Improvements measured at this same volume.

Rate-limiter caveat: the dev `apiLimiter` (1000 req/min) was bypassed for this run via a `SHIPSHAPE_AUDIT=1` env flag wired into `api/src/app.ts`. Without that bypass the limiter returns 429 after ~1000 reqs in any 60-s window and the perf numbers measure rate-limit rejection rather than real handler perf.

Latency (ms) under autocannon (10-second runs, JSON: [raw/perf/](#)):

Endpoint	Conn	RPS avg	P50	P90	P97.5	P99	Max	Bytes/s
/api/auth/me (377 B)	10	488	18	28	36	41	348	735 KB
/api/auth/me	25	588	41	51	62	68	87	886 KB
/api/auth/me	50	204	136	515	1174	1575	2785	307 KB
/api/documents?type=wiki (2.4 KB)	10	489	17	29	52	68	177	1.7 MB
/api/documents?type=wiki	25	596	34	73	104	127	193	2.1 MB
/api/documents?type=wiki	50	676	68	113	181	222	462	2.4 MB
/api/issues (102 KB)	10	142	57	104	128	165	235	14 MB
/api/issues	25	132	183	219	247	274	324	13 MB
/api/issues	50	136	354	403	486	515	560	14 MB
/api/dashboard/my-work (6.9 KB)	10	369	26	34	44	60	135	2.9 MB
/api/dashboard/my-work	25	231	91	162	278	318	365	1.8 MB
/api/dashboard/my-work	50	379	125	162	193	246	346	3.0 MB

/api/weeks (4.3 KB)	10	396	24	31	37	44	112	2.1 MB
/api/weeks	25	449	53	63	83	122	200	2.4 MB
/api/weeks	50	470	104	117	130	147	249	2.5 MB

Findings

1. `/api/issues` **over-fetches** `content` (full TipTap doc) for every issue. `issues.ts:126` selects `d.content`. Response is **~1 KB/issue** when titles alone would be ~150 B. At 104 issues this is 102 KB; at 1000 issues it'd be ~1 MB per call. P99 already 165 ms at `c=10` with this much data. **The single biggest perf win in the app.** (Severity: high.)
2. **Connection pool saturates at `c=50`.** `api/src/db/client.ts` sets `max: 10` for dev (`max: 20` for prod). At 50 concurrent in-flight requests, ≥ 40 are queued for a Pool slot. `/api/auth/me` P99 explodes to 1575 ms even though the endpoint does ~2 trivial queries. Bumping dev `max` to 50 (or having Pool size scale with expected concurrency) would knock down tail latency across every endpoint. (Severity: high.)
3. `/api/dashboard/my-work` **has non-monotonic latency:** $44 \rightarrow 278 \rightarrow 193$ ms P97.5 across `c=10/25/50`. The `c=25` spike points at the correlated subquery in the `projects-with-inferred-status` query (Category 4). (Severity: medium.)
4. **No LIMIT / pagination** on `/api/issues` or `/api/documents`. Latency scales linearly with seed size; will degrade hard at $10\times$ docs. (Severity: high — scaling cliff.)

Improvement target

-20% P95 on ≥ 2 endpoints under identical conditions.

Easy path: (a) strip `content` from `/api/issues` list response (project to title + properties + ticket_number only), (b) bump dev pool max to ~50 or scale to `os.availableParallelism() * 4`. Either alone should hit the target on multiple endpoints.

Category 4 — Database Query Efficiency

Methodology

- Enable `log_statement = 'all'` and `log_duration = on` on the dev Postgres for the measurement window. For native Windows Postgres, set in `postgresql.conf` and `pg_ctl reload`.
- Drive five user flows from the UI; capture queries from the log.
- `EXPLAIN (ANALYZE, BUFFERS)` on the top-5 slowest distinct queries.
- Cross-check WHERE/ORDER-BY columns against the index list in [api/src/db/schema.sql](#) (38 indexes; pretty thorough already).

Baseline (EXPLAIN ANALYZE on suspect queries)

Captured via `psql` against the live seeded DB. Full output: [raw/explain-analyze.txt](#).

At seeded volume (104 issues, 250 docs total), every plan executes in 1–2 ms. That sounds great, but two observations qualify it:

- **All filtered queries use Seq Scan + Filter.** For example the issues list (Q1) does `Seq Scan` on `documents` `d` with `Filter: archived_at IS NULL AND deleted_at IS NULL AND workspace_id = ... AND document_type = 'issue'`. Rows removed by filter: 153 (out of 257 docs). At 1 ms/scan and 257 rows that's fine — at 100,000 docs the seq scan dominates. The `idx_documents_active` partial index on

(workspace_id, document_type) WHERE archived_at IS NULL AND deleted_at IS NULL exists but isn't being chosen — the planner thinks the seq scan is cheaper at this scale, and at 250 rows it is.

- **The dashboard projects query (Q2) reads 596 shared buffers.** Plan time alone: 9.4 ms (only 2 ms execution). 596 buffer hits for 15 projects = ~40 buffers per project. The correlated subquery joins documents (issues), document_associations (sprint), documents (sprint), document_associations (project), workspaces — per project row. Indexes hide the cost today; at scale this will grow $N \times M$ where N =projects and M =issues-per-project.

#	Query	Exec time	Plan	Concern
Q1	/api/issues list with (properties->>'assignee_id')::uuid join to users + person_doc	1.339 ms	Seq Scan + 2 Hash Left Joins	Will be Seq Scan at 10x — needs functional index on properties->>'assignee_id'
Q2	/api/dashboard/my-work projects-with-inferred-status (correlated subquery)	2.054 ms	Nested Loop + Bitmap Heap Scan	Correlated subquery scans all issues per project; quadratic at scale
Q3	/api/dashboard/my-work issues filtered by assignee_id + state NOT IN ('done', 'cancelled')	< 1 ms	(output truncated)	Same JSONB-cast pattern; would benefit from same functional index

Findings

1. **Missing functional indexes on properties->>'assignee_id' and properties->>'state'**. The GIN index on the whole properties JSONB column [exists](#) but PostgreSQL doesn't use a GIN index for equality on a single extracted text key efficiently. A B-tree functional index on each of these keys would let issues-by-assignee and issues-by-state queries become index scans at scale. (idx_documents_person_user_id is exactly this pattern, on a different key.) (Severity: high at scale; low at current volume.)
2. **Correlated subquery in projects-with-inferred-status** ([dashboard.ts:153-181](#)). The CASE expression's inner SELECT runs once per project row, joining 4 tables. Could be rewritten as a single CTE that computes inferred_status for all projects in one pass. (Severity: medium — quadratic growth path.)
3. **/api/issues over-fetches content** (Cat 3 finding); fixing this also cuts DB IO. (Severity: high.)
4. **/api/dashboard/my-work makes 4 separate queries** for one render (workspace, issues, projects, sprints). Could be one query with UNION ALL or one CTE. Saves 3 round-trips per page load. (Severity: medium — easy 25-30% query count reduction.)

Improvement target

-20% query count on a flow OR -50% on slowest query.

Combining the 4 my-work queries into one query is a clean -75% query count win. The functional indexes will become measurable at higher data volume.

Improvement target

-20% query count on a flow OR -50% on slowest query.

Category 5 — Test Coverage & Quality

Methodology

- File counts via `Glob` .
- Test counts via ripgrep over `^\s*(test|it)(\.fixme|\.skip|\.only)?\s*\s*` (counts each test entry, including ones inside `describe`).
- Suite runtime + flakes: requires E2E run. Will use the `/e2e-test-runner` skill per CLAUDE.md and capture `test-results/summary.json` over 3 runs to detect flakes.
- Coverage: configure `vitest --coverage` for api and web (not currently enabled). Report v8 line/branch %.

Baseline

Metric	Value
Total E2E spec files	71 (brief says "73+"; file count, not test count)
Total E2E test entries	866
Total API unit test files	28
Total API unit tests	447
Total Web unit test files	16
Total Web unit tests	151
Grand total tests	~1,464
<code>.fixme / .skip / .only</code> markers (e2e)	0
Pre-commit empty-test hook	yes (<code>scripts/check-empty-tests.sh</code> per CLAUDE.md)
API unit test status (no DB)	28/28 files fail to start — <code>ECONNREFUSED</code> from pg-pool. All 451 tests "skipped" because <code>beforeAll</code> in <code>api/src/test/setup.ts:14</code> issues <code>TRUNCATE CASCADE</code> and pg retries until exhaustion. 3248s (54 min) burned on retries.
Pass / Fail / Flake rates	<i>(pending live Postgres + run)</i>
Suite runtime	<i>(pending)</i>
Code coverage %	<i>(pending; tooling not configured)</i>

Raw vitest output: [raw/api-vitest-baseline.txt](#).

Findings (preliminary)

- **Coverage tooling not configured.** `@vitest/coverage-v8` is not in any package's devDependencies — even though `api/package.json` has a `test:coverage` script that calls `vitest run --coverage` .

Configuring is itself a measurable improvement worth at least mentioning.

- **No Playwright sharding for CI** in `playwright.config.ts` . With 866 tests at 4 workers × 60s avg, CI time is non-trivial. (This is about quality of the suite infra, not test count.)

Improvement target

+3 meaningful tests on previously-uncovered critical paths OR fix 3 flakes with documented RCA. Each test must include a comment explaining the regression it catches.

Critical flows to check coverage of

- Session timeout & re-auth (sliding window + 12-hour absolute).
- Yjs offline edit → online merge.
- API token auth (Bearer) bypassing CSRF.
- Workspace membership revoke — does the user actually get kicked out?
- Document conversion (issue ↔ project) preserving `original_type` / `conversion_count` .

Category 6 — Runtime Error & Edge-Case Handling

Methodology

- Open the dev app in Chrome with DevTools open. Use the app through the 4 golden flows. Note every console error and warning.
- Drop the network in DevTools while editing a doc collaboratively (use 2 browser profiles). Re-enable. Verify Yjs convergence and UI recovery.
- Submit malformed input on each form: empty, 10k-char string, `<script>alert(1)</script>` , control chars, emoji.
- Two-tab same-field-edit. Verify CRDT convergence.
- Throttle to "Slow 3G" in DevTools. Note hanging spinners / silent failures.

Baseline (live probes)

Malformed-input probe results against `POST /api/issues` (raw: [raw/malformed/issues-post.txt](#)):

Input	Result	Concern
Empty body	400 JSON with zod details	✅ Correct
Non-JSON body ("this is not JSON")	400 HTML with full Node.js stack trace including at <code>JSON.parse</code> (<code><anonymous></code>) at <code>createStrictSyntaxError (...)</code>	Stack trace leak — Express's default error handler escapes through
<code><script>alert(1)</script></code> as title	201 Created , stored verbatim	Needs verification that no downstream <code>dangerouslySetInnerHTML</code> consumes title (likely OK due to React's default escaping)
estimate: -99999 (not in <code>createIssueSchema</code>)	201 Created , field silently dropped	Zod schema isn't <code>.strict()</code> — accepts arbitrary extra fields
title: <code>''; DROP TABLE documents; --</code>	201 Created , stored as literal text	✅ Parameterized queries prevent injection

Deep nested 10 KB object	201 Created	✔ JSON parser handled it fine
--------------------------	-------------	-------------------------------

Console errors from the a11y walk (which doubles as a Cat 6 probe): `/login` emitted 1 console error during navigation; all other 11 routes were silent. Detail in each `raw/a11y/<route>.json` under `consoleErrors`.

Existing error-handling infrastructure (preliminary read)

The codebase already has solid foundations:

- `MutationCache.onError` in [web/src/lib/queryClient.ts:166](#) emits via `notifyMutationError` → `<MutationErrorToast>` in `main.tsx`.
- `subscribeToCacheCorruption` detects bad IndexedDB state and notifies listeners.
- `handleSessionExpired` in `web/src/lib/api.ts` redirects to `/login?expired=true&returnTo=...`.
- `ErrorBoundary` exists at [web/src/components/ui/ErrorBoundary.tsx](#) and is used in `pages/App.tsx` and `components/Editor.tsx`. Has a "Try Again" reset path. **Open question for live audit:** is the boundary at the right granularity? E.g., does an error inside a route render gate the whole `AppLayout`?

Concrete gaps already identified (pre-live-app)

1. **API tests slow-fail when Postgres is unreachable.** [api/src/test/setup.ts:14](#) calls `pool.query(TRUNCATE ...)` directly in `beforeAll`. There is no connection precheck and no `connectionTimeoutMillis` set on the `Pool`. Result observed during this audit: 28/28 test files spend a combined **3248 seconds (54 minutes)** retrying connections before failing the whole suite. **Severity: medium** — affects developer experience; doesn't ship to users but a clear quality-of-life win.
2. **Pre-commit `check-empty-tests.sh` is a false-positive generator.** The awk parser at [scripts/check-empty-tests.sh:42-54](#) treats the first `^\s*\}\);` it finds as the end of a `test()` body. When a test contains a nested arrow function (e.g. `await context.route('...', async (route) => { ... });`), the inner `});` is matched first — before any `expect()` or `page.` call inside the outer test runs through the regexes. **Effect:** tests with real content are flagged as "empty," and developers either resort to `--no-verify` (explicitly forbidden by `CLAUDE.md`) or are blocked. **Confirmed instances on master:** `autosave-race-conditions.spec.ts:179, 277` (both have `context.route` + real `expect`); `critical-blockers.spec.ts:118, 143` (similar); `session-timeout.spec.ts:505, 1099`. **Severity: high** — actively blocks commits; encourages bypass; ironic for a quality-gate hook.

Hypotheses

- No `<ErrorBoundary>` wrapping `AppLayout` → an unhandled render error blanks the whole UI.
- Slow-3G probably exposes one of: dashboard query timing out without indicator, save-on-blur silently failing on a flaky network, or missing optimistic update for an obvious action.
- Malformed input on `properties` JSONB endpoints is only zod-validated on known keys (`.passthrough` patterns) — XSS via wiki content is rendered, so TipTap's sanitizer is the only line of defense.

Improvement target

3 error-handling gaps fixed, ≥1 involving real user-facing data loss or confusion. Each fix needs repro steps + before/after.

Category 7 — Accessibility

Methodology

- Run Lighthouse (Chrome built-in) on every major route while signed in: `/my-week` , `/docs` , `/documents/:id` , `/issues` , `/programs` , `/team/allocation` , `/team/directory` , `/team/status` , `/team/org-chart` , `/settings` .
- Run `@axe-core/playwright` programmatically against the same set; save violations as JSON to `shipshape/audit/raw/axe-<route>.json` . Categorize by impact (Critical / Serious / Moderate / Minor).
- Keyboard nav: Tab through each route. Note any focus trap, lost focus, or unreachable element.
- Screen reader: NVDA on Windows (free). Verify landmarks, headings, and form labels.
- Color contrast: spot-check with the WCAG contrast checker on suspect tokens in the design system (theming via Tailwind + USWDS).

Baseline (axe-core scan via [shipshape/audit/axe-scan.spec.ts](#))

12 routes scanned via Playwright + `@axe-core/playwright` with WCAG 2 A/AA + WCAG 2.1 A/AA tags. Raw per-route JSON: [raw/a11y/](#).

Route	Total	Critical	Serious	Moderate	Minor	Console
<code>/dashboard</code>	1	0	1	0	0	0
<code>/docs</code>	0	0	0	0	0	0
<code>/issues</code>	0	0	0	0	0	0
<code>/login</code>	0	0	0	0	0	1
<code>/my-week</code>	1	0	1	0	0	0
<code>/programs</code>	0	0	0	0	0	0
<code>/projects</code>	1	0	1	0	0	0
<code>/settings</code>	0	0	0	0	0	0
<code>/team/allocation</code>	1	0	1	0	0	0
<code>/team/directory</code>	0	0	0	0	0	0
<code>/team/org-chart</code>	0	0	0	0	0	0
<code>/team/status</code>	1	0	1	0	0	0

Single violation rule across all 5 hit routes: `color-contrast` (impact: serious; WCAG 2 AA 1.4.3). 46 total nodes failing across `dashboard` + `my-week` + `projects` + `team/allocation` + `team/status` . **The README claims WCAG 2.1 AA conformance — this is a direct, verifiable contradiction.**

Existing infrastructure (preliminary read)

- USWDS tokens via `@uswds/uswds` — gives a reasonable a11y default.
- `@axe-core/playwright` is already a devDep (root `package.json`).
- Several E2E specs already exercise a11y: `accessibility.spec.ts` (11 tests), `accessibility-remediation.spec.ts` (57 tests — large!), `check-aria.spec.ts` , `status-colors-accessibility.spec.ts` . The team has invested here. **This means baselines might be good — and the improvement target may need to focus on raising bar (Lighthouse score) rather than fixing critical violations (might already be at zero).**

- Recent commits (last week) include `838375e` fix: use `aria-label` instead of `aria-labelledby` for USWDS Icon `a11y` — active a11y work.
- **Zero** `<div onClick=>` or `<span onClick=>` patterns in `web/src/**/*.tsx` (positive finding from static grep). The codebase uses proper `<button>` elements for interactive elements. One major class of WCAG 2.1.1 (Keyboard) violations is structurally avoided.

Improvement target

+10 Lighthouse points on lowest-scoring page OR fix all Critical/Serious violations on top 3 pages.

Status

All 7 categories baselined. No outstanding blockers.

How to reproduce

```
# 1. Postgres 18 running locally on :5432; ship_dev DB owned by ship user
# 2. From repo root:
Copy-Item api\.env.example api\.env.local # if not already
corepack pnpm install
corepack pnpm db:migrate
corepack pnpm db:seed
# Start API with audit-mode env to bypass the dev rate limiter
$env:PORT="3000"; $env:CORS_ORIGIN="http://localhost:5173"; $env:SHIPSHAPE_AUDIT="1"
corepack pnpm --filter @ship/api dev # leave running
# In another shell:
$env:VITE_PORT="5173"; $env:API_PORT="3000"
corepack pnpm --filter @ship/web dev # leave running
```

Then re-run measurements:

- **Cat 1:** Grep patterns from § 1 above; `cd web && node ./node_modules/typescript/bin/tsc --noEmit --project tsconfig.strict-probe.json`
 - **Cat 2:** `cd web && ANALYZE=1 VITE_API_URL= node ./node_modules/vite/bin/vite.js build → dist/bundle-stats.html`
 - **Cat 3:** Get a Bearer token via `POST /api/api-tokens`, then `npx autocannon -c {10,25,50} -d 10 -H "Authorization: Bearer ..."` -j `http://localhost:3000/<endpoint>` for each of the 5 endpoints.
 - **Cat 4:** Paste the 3 queries in `raw/explain-analyze.txt` into psql with `EXPLAIN (ANALYZE, BUFFERS)`.
 - **Cat 6:** Replay the 7-input probe from `raw/malformed/issues-post.txt`.
 - **Cat 7:** `npx playwright test --config=shipshape/audit/axe-playwright.config.ts`.
-

Provenance

- Repository commit at audit time: `076a183` (master HEAD when branched).
- Audit branch: `shipshape/audit`.
- All raw measurement outputs go in [raw/](#) and are committed (decided in kickoff Q&A).
- This report is updated incrementally as numbers come in; the Tuesday deadline version has every "*(pending)*" replaced with concrete numbers.